🍽️ Restaurant Order Management System

📖 Project Overview

The Restaurant Order Management System is a relational database project developed using MySQL. This project is designed to manage restaurant operations such as customers, employees, menu items, orders, order items, and payments.

The database demonstrates the implementation of:

- ER diagram design (first step of the project)
- Table creation
- Primary and foreign keys
- Data insertion
- Relationships between tables
- Indexes, views, triggers, and stored procedures
- SQL queries of different difficulty levels

✨ Features

- Proper relational database design with 7 tables (6 core tables and 1 audit table)
- Use of primary and foreign keys with CHECK and UNIQUE constraints
- One row per menu item in each order, with quantity updates handled automatically
- Stored procedures with input validation and clear error messages
- Audit log for every insert, update, and delete on order items
- Totals that stay accurate when items and payments are joined (no inflated sums)
- ER Diagram included
- Sample records inserted for testing
- Query categorization based on difficulty level

🛠️ Technologies Used

- MySQL 8.0.16+
- SQL

🗺️ ER Diagram

The project began with designing the ER (Entity Relationship) Diagram, which identified the entities, attributes, and relationships. The database tables were created from this design.

ER Diagram Link: View ER Diagram

ER Diagram

🗄️ Database Tables

The project contains the following tables:

```
restaurantdb
├── Customers
├── Employees
├── Menu_Items
├── Orders
├── Order_Items
├── Payments
└── order_items_log
```

🔗 Relationships

- One customer can place multiple orders
- One employee can handle multiple orders
- One order can contain multiple menu items through Order_Items
- One menu item can appear in many different orders
- One order can have multiple payments
- Order_Items connects orders and menu items, and each menu item appears only once per order
- order_items_log is an audit table filled automatically by triggers and has no foreign keys

⚙️ Database Objects

⚡ Indexes

Indexes are created on frequently searched and filtered columns of orders, order items, payments, and menu items. Their purpose is to speed up lookups, filtering, and sorting, so queries run faster as the data grows.

👁️ Views

Views provide ready-made reports without rewriting complex joins each time.

- `view_order_details` - order with customer, employee, and order total
- `view_category_averages` - average price and item count per menu category

- `view_customer_totals` - order count and total order value per customer (cancelled orders excluded)
- `view_payment_status` - payments with customer names
- `view_sales_summary` - order total compared with the amount actually paid

🔔 Triggers

Triggers enforce business rules and keep data consistent automatically.

- Validation triggers - reject zero or negative quantity and negative unit price (on insert and update)
- Duplicate prevention trigger - stops the same menu item from being added twice to one order
- Audit triggers - record every insert, update, and delete on order items in the audit log

📦 Stored Procedures

Stored procedures handle the main workflows safely, with validation and clear error messages.

- `sp_get_customer_orders` - lists all orders of a customer with totals
- `sp_add_order_item` - adds a menu item to an order, or increases its quantity if already present; rejects cancelled orders and unavailable items
- `sp_category_average` - shows the average price for a menu category
- `sp_order_report` - gives a line-by-line report for one order
- `sp_customer_report` - summarizes a customer's order count and total order value

📝 SQL Queries Included (Q1 to Q21)

🟢 Easy Level Queries (Q1, Q2, Q3, Q10, Q21)

Queries using:

- SELECT and DISTINCT
- WHERE clause
- Date range filtering
- ORDER BY
- LIMIT and OFFSET (second-highest price)

🟡 Intermediate Level Queries (Q5, Q6, Q8, Q11, Q14, Q15)

Queries using:

- Aggregate Functions
- COUNT()
- MAX()

- AVG()
- GROUP BY
- HAVING

🔴 Hard Level Queries (Q4, Q7, Q9, Q12, Q13, Q16, Q17, Q18, Q19, Q20)

Queries using:

- INNER JOIN and LEFT JOIN
- Multi-table joins with GROUP BY
- Subqueries (correlated and derived tables)
- CASE expressions
- COALESCE and string functions (CONCAT)
- Anti-join to find customers with no orders

▶️ How to Run

1. Install MySQL 8.0.16 or later and open MySQL Workbench.
2. Open the project SQL script (`restaurant_db.sql`).
3. Execute the script from beginning to end.
4. Verify the tables, sample data, views, triggers, and stored procedures.

> ⚠️ The script begins with `DROP DATABASE IF EXISTS restaurantdb;`, which deletes any existing database with that name.

🎓 Learning Outcomes

This project helped in understanding:

- Database normalization
- SQL query writing
- Table relationships
- Aggregate functions
- Joins and relational analysis
- ER diagram design
- Views, triggers, and stored procedures
- Data validation and audit logging

🔮 Future Scope

- **Full-Stack Integration:** Connect the database to a Node.js backend and a responsive React frontend for taking orders.
- **Inventory Tracking:** Add stock levels for menu items and update them automatically as orders are placed.
- **Delivery Management:** Track delivery partners, addresses, and delivery status for delivery orders.
- **Advanced Analytics:** Integrate Power BI or Tableau for live sales dashboards.

👩‍💻 Author

<Your Name>